

Recommendation Engine Architecture

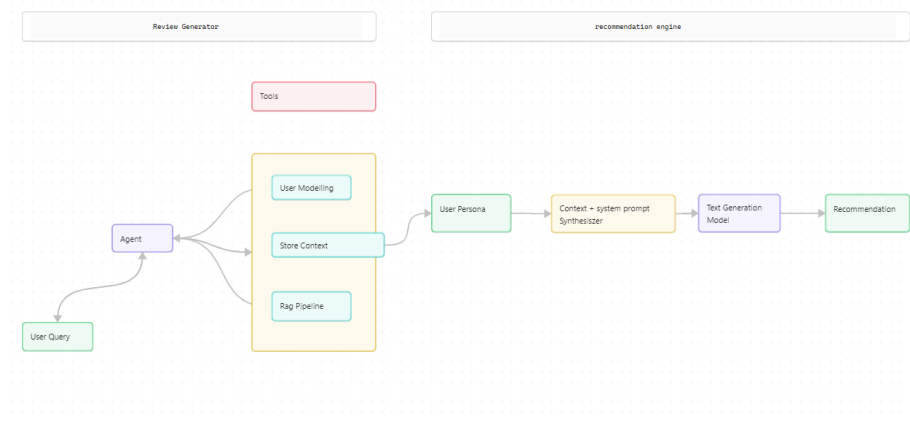


Figure 1: Architecture diagram

Overview

This diagram describes a two-part system:

1. **Review Generator** on the left, which captures a user query, routes it through an agent, and uses tools to transform raw interaction data into structured user understanding.
2. **Recommendation Engine** on the right, which consumes the structured user persona, synthesizes the final prompt context, sends it to a text generation model, and produces the recommendation output.

The central design idea is separation of concerns:

- **Capture and structure user signals first**
- **Represent the user as a persona**
- **Assemble prompt context explicitly**
- **Generate the recommendation only after the context is ready**

This makes the system easier to extend, debug, and tune.

What the diagram shows

The flow starts with a **User Query**. The query is received by an **Agent**, which acts as the orchestration layer. From there, the agent can invoke a set of **Tools**. Those tools are grouped around three jobs:

- **User Modelling**
- **Store Context**
- **RAG Pipeline**

The output of these tools is a **User Persona**. That persona is then passed into a **Context + system prompt Synthesizer**, which prepares the exact model input. The prepared prompt is then sent to a **Text Generation Model**, and the model returns the final **Recommendation**.

Component breakdown

1. User Query

The user query is the raw input into the system. It may contain: - a request for a recommendation - preference signals - product or content constraints - implicit behavioral cues

The query is not used directly as the final model input. It is first interpreted and normalized.

2. Agent

The agent is the control layer. Its job is to: - receive the query - decide which tools to call - combine tool outputs - route the processed information into the recommendation pipeline

In this architecture, the agent is not the recommender itself. It is the orchestrator.

3. Tools

The tools block represents the utility layer used by the agent. The diagram shows three main tool functions.

User Modelling This module converts raw signals into a structured representation of the user. It can infer: - preferences - recurring behavior - interest patterns - intent - likely response style

The purpose is to move from unstructured signals to a stable persona representation.

Store Context This module persists relevant session or historical context. It is responsible for: - retaining previous interactions - preserving long-term preference signals - making prior context available to later stages

This prevents the system from treating every query as isolated.

RAG Pipeline This module retrieves external or internal knowledge that may improve the response. Typical uses include: - fetching relevant past interactions - retrieving product information - pulling supporting evidence or contextual memory

The retriever provides grounding so the final recommendation is not based only on the current query.

4. User Persona

The user persona is the structured output of the modelling layer. It is the system's compact representation of the user.

A good persona can include: - preference summaries - behavioral tendencies - sentiment patterns - confidence-weighted signals - exclusions or dislikes - recency-sensitive context

This is the main input to the recommendation engine.

5. Context + system prompt Synthesizer

This block prepares the final prompt for the text generation model.

Its job is to combine: - the user persona - retrieved context - task instructions - output formatting rules - any safety or relevance constraints

This step is important because prompt quality strongly affects generation quality. The synthesizer ensures that the model receives a controlled, structured, and task-specific prompt instead of a loose concatenation of fields.

6. Text Generation Model

This is the language model that turns the prepared prompt into the actual recommendation text.

It should: - follow the instruction format precisely - stay grounded in the persona and context - avoid inventing unsupported claims - produce output in the requested schema or natural language format

The model is the generation layer, not the reasoning memory layer.

7. Recommendation

This is the final output of the system. It can take the form of: - a single recommended item - ranked options - a recommendation sentence - a structured JSON response for downstream applications

The exact output format depends on the product requirements.

End-to-end flow

A practical reading of the diagram is:

1. The user submits a query.
2. The agent receives the query.
3. The agent calls tools for user modelling, context storage, and retrieval.

4. Those tools produce a structured persona and related context.
5. The persona is passed to a prompt synthesizer.
6. The synthesizer constructs the final system prompt and task prompt.
7. The text generation model produces the recommendation.
8. The recommendation is returned to the user.

Why this architecture is useful

Modularity

Each block has one responsibility. That makes the system easier to maintain and debug.

Better personalization

Recommendations are driven by a persona rather than a single isolated query.

Context retention

Stored context makes the system more consistent across multiple interactions.

Retrieval grounding

A RAG pipeline reduces blind generation and improves factual alignment.

Prompt control

A dedicated synthesizer gives precise control over how the model is instructed.

Easier iteration

Each stage can be improved independently: - persona quality - retrieval quality
- prompt construction - generation quality - storage strategy

Practical implementation interpretation

A likely implementation pattern is:

- The **agent** receives the request.
- The **user modelling** component extracts features from the query and history.
- The **store context** component writes relevant state to memory or a database.
- The **RAG pipeline** retrieves useful supporting context.
- The **persona builder** consolidates those signals into a stable user profile.
- The **prompt synthesizer** assembles the final prompt.
- The **text generation model** produces the recommendation response.

Notes on the diagram labels

- **Review Generator** appears to describe the left-side processing and preparation stage.
- **Recommendation Engine** appears to describe the right-side generation stage.
- The diagram emphasizes that the recommendation is not produced directly from the user query. It is produced after modelling, storage, retrieval, and prompt synthesis.

Summary

The diagram presents a recommendation workflow built around structured persona creation, context retention, retrieval augmentation, and prompt synthesis. The architecture is suitable for systems that need personalized outputs while keeping the generation stage controlled and explainable at a systems level.